3:25 ··· 120 KB/s 4G+ 46%

Instagram

techkanya
Travis Scott · FE!N (feat. Playboi Carti) E

5/10

Problem: Merge overlapping intervals.P
Solution:

```python
intervals.sort(key=lambda x: x[0])

merged = [intervals[0]]

for s, e in intervals[1:]:

  if s <= merged[-1][1]:

    merged[-1][1] = max(merged[-1][1], e)

  else:

    merged.append([s, e])
```

---

9. Container With Most Water (LeetCode 11)

Problem: Find the maximum water that can be trapped between two lines.
Solution:

```python
l, r = 0, len(h)-1

best = 0

while l < r:

  area = min(h[l], h[r]) * (r - l)

  best = max(best, area)

  if h[l] < h[r]: l += 1

  else: r -= 1
```

---

10. Rotate Image (LeetCode 48)

Problem: Rotate a matrix 90 degrees clockwise.
Solution:

```Python
matrix[:] = zip(*matrix[::-1])
```

2. Strings (10 Questions)

1. **20. Valid Parentheses**
2. **125. Valid Palindrome**
3. **242. Valid Anagram**

2,698 442 159

techkanya 🚀 Master DSA with 75 Must-Solve LeetCode Questions!

Want to crack coding interviews at top tech companies? 💻

3:24 ··· 30.3 KB/s 4G+ 47%

Instagram

techkanya
Travis Scott · FE!N (feat. Playboi Carti) E

75 DSA Questions from Leet-Code

1. Arrays (10 Questions)

1. **1. Two Sum**
2. **121. Best Time to Buy and Sell Stock**
3. **88. Merge Sorted Array**
4. **217. Contains Duplicate**
5. **238. Product of Array Except Self**
6. **53. Maximum Subarray**
7. **15. 3Sum**
8. **56. Merge Intervals**
9. **11. Container With Most Water**
10. **48. Rotate Image**

Solution:

1. Two Sum (LeetCode 1)

Problem: Find two numbers in the array that add up to a specific target.
Solution:

```
python
for i, n in enumerate(nums):
    diff = target - n
    if diff in seen:
        return [seen[diff], i]
    seen[n] = i
```

---

2. Best Time to Buy and Sell Stock (LeetCode 121)

Problem: Maximize profit by choosing one day to buy and another to sell.
Solution:

```
python
min_price, max_profit = inf, 0
for p in prices:
    min_price = min(min_price, p)
```

2,698 442 159

techkanya 🚀 Master DSA with 75 Must-Solve LeetCode Questions!

Want to crack coding interviews at top tech companies? 💻

3:25 ··· 120 KB/s 4G+ 46%

Instagram

techkanya
Travis Scott · FE!N (feat. Playboi Carti) E

6/10

4. **49. Group Anagrams**
5. **5. Longest Palindromic Substring**
6. **76. Minimum Window Substring**
7. **28. Find the Index of the First Occurrence in a String**
8. **443. String Compression**
9. **14. Longest Common Prefix**
10. **459. Repeated Substring Pattern**

Solution:

1. Valid Parentheses (LeetCode 20)

```python
stack = []

pairs = {')': '(', ']': '[', '}': '{'}

for c in s:

  if c in pairs:

    if not stack or stack.pop() != pairs[c]: return False

  else:

    stack.append(c)

return not stack
```

---

2. Valid Palindrome (LeetCode 125)

```python
filtered = [c.lower() for c in s if c.isalnum()]

return filtered == filtered[::-1]
```

---

3. Valid Anagram (LeetCode 242)

```python
return Counter(s) == Counter(t)
```

---

4. Group Anagrams (LeetCode 49)

```python
res = defaultdict(list)

for w in strs:
```

2,698 442 159

techkanya 🚀 Master DSA with 75 Must-Solve LeetCode Questions!

Want to crack coding interviews at top tech companies? 💻

3:25 ··· 54.5 KB/s 4G+ 46%

Instagram

techkanya
Travis Scott · FE!N (feat. Playboi Carti) E

8. String Compression (LeetCode 443)

```python
i = write = 0
while i < len(chars):
    ch, count = chars[i], 0
    while i < len(chars) and chars[i] == ch:
        i += 1; count += 1
    chars[write] = ch; write += 1
    if count > 1:
        for c in str(count):
            chars[write] = c; write += 1
```

---

9. Longest Common Prefix (LeetCode 14)

```python
prefix = strs[0]
for s in strs[1:]:
    while not s.startswith(prefix):
        prefix = prefix[:-1]
return prefix
```

---

10. Repeated Substring Pattern (LeetCode 459)

```Python
return s in (s + s)[1:-1]
```

Linked Lists (8 Questions)

1. **206. Reverse Linked List**
2. **21. Merge Two Sorted Lists**
3. **19. Remove Nth Node From End of List**
4. **141. Linked List Cycle**
5. **2. Add Two Numbers**
6. **160. Intersection of Two Linked Lists**
7. **234. Palindrome Linked List**
8. **25. Reverse Nodes in k-Group**

2,698 442 159

techkanya 🚀 Master DSA with 75 Must-Solve LeetCode Questions!

Want to crack coding interviews at top tech companies? 💻

3:25 ··· 259 KB/s 4G+ 46%

Instagram

abhi_techhub
KALEO · Way down We Go

insta @abhi_techhub.

MOST ASKED DSA INTERVIEW QUESTIONS

(0-3 YEARS) (5-20 LPA) PAGE 1

1. ARRAYS – MOST ASKED QUESTIONS

★ Arrays are the foundation of DSA. Most of the interview questions for 0-3 YOE are based on Arrays.

#	QUESTION	PATTERN	EXPECTED COMPLEXITY	KEY IDEA / HINT
1	Two Sum (LeetCode 1)	Hashing	O(n) time O(n) space	• Use a HashMap to store number → index • For each number, check if target - num exists
2	Best Time to Buy & Sell Stock (LC 121)	One Pass / Greedy	O(n) time O(1) space	• Keep track of minimum price so far • Max profit = max(sell - min_buy)
3	Maximum Subarray (Kadane's Algo) (LC 53)	Kadane's Algorithm	O(n) time O(1) space	• Keep running sum • If sum < 0, reset to 0 • Update max_sum
4	Move Zeroes (LC 283)	Two Pointers	O(n) time O(1) space	• Place all non-zero elements at front • Fill remaining with 0
5	Majority Element (LC 169)	Boyer-Moore Voting Algo	O(n) time O(1) space	• Count / cancel pairs • Candidate will be the majority
6	Merge Sorted Arrays (LC 88)	Two Pointers	O(n + m) time O(1) space	• Start from the end • Place larger element at the end of nums1
7	Product of Array Except Self (LC 238)	Prefix & Suffix	O(n) time O(1) space (ignoring output)	• Left product & Right product • res[i] = left[i] * right[i]

★ IMPORTANT PATTERNS TO RECOGNIZE

→ Hashing Pattern
→ Subarray / Kadane's Pattern
→ Two Pointers Pattern
→ Prefix / Suffix Pattern
→ Sliding Window Pattern
→ Sorting / Greedy Pattern

1,966 822 125

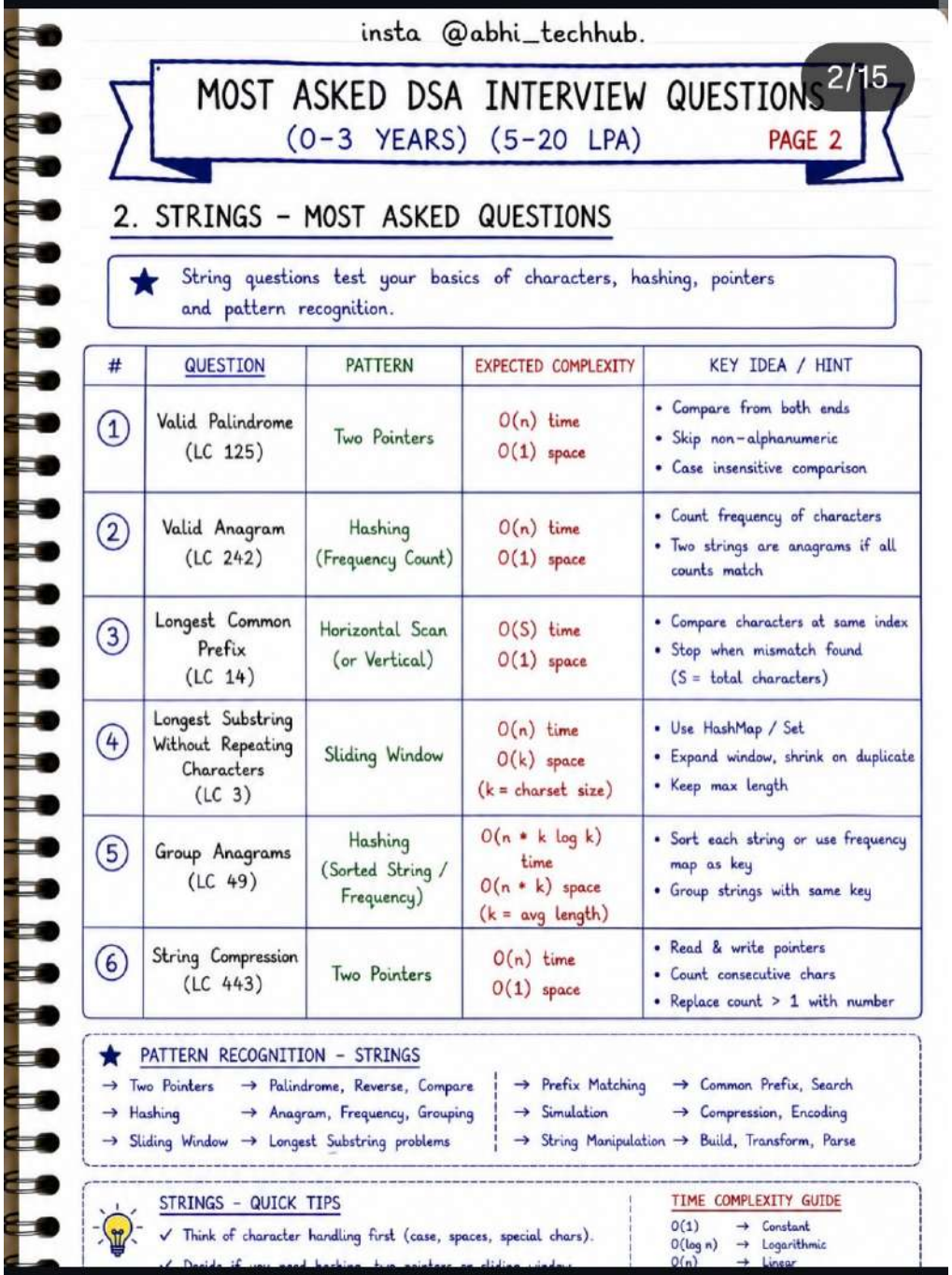
Liked by _.lalit._ and others

abhi_techhub 🔥 Preparing DSA for 5–20 LPA interviews? Don't solve 500 random questions. Master the patterns that keep repeating.

3:25 ··· 2.64 KB/s 4G+ 46%

Instagram

abhi_techhub
AI info



insta @abhi_techhub.

2/15

MOST ASKED DSA INTERVIEW QUESTIONS
(0-3 YEARS) (5-20 LPA) PAGE 2

2. STRINGS - MOST ASKED QUESTIONS

★ String questions test your basics of characters, hashing, pointers and pattern recognition.

#	QUESTION	PATTERN	EXPECTED COMPLEXITY	KEY IDEA / HINT
1	Valid Palindrome (LC 125)	Two Pointers	O(n) time O(1) space	• Compare from both ends • Skip non-alphanumeric • Case insensitive comparison
2	Valid Anagram (LC 242)	Hashing (Frequency Count)	O(n) time O(1) space	• Count frequency of characters • Two strings are anagrams if all counts match
3	Longest Common Prefix (LC 14)	Horizontal Scan (or Vertical)	O(S) time O(1) space	• Compare characters at same index • Stop when mismatch found (S = total characters)
4	Longest Substring Without Repeating Characters (LC 3)	Sliding Window	O(n) time O(k) space (k = charset size)	• Use HashMap / Set • Expand window, shrink on duplicate • Keep max length
5	Group Anagrams (LC 49)	Hashing (Sorted String / Frequency)	O(n * k log k) time O(n * k) space (k = avg length)	• Sort each string or use frequency map as key • Group strings with same key
6	String Compression (LC 443)	Two Pointers	O(n) time O(1) space	• Read & write pointers • Count consecutive chars • Replace count > 1 with number

★ PATTERN RECOGNITION - STRINGS

→ Two Pointers → Palindrome, Reverse, Compare
→ Hashing → Anagram, Frequency, Grouping
→ Sliding Window → Longest Substring problems
→ Prefix Matching → Common Prefix, Search
→ Simulation → Compression, Encoding
→ String Manipulation → Build, Transform, Parse

STRINGS - QUICK TIPS

✓ Think of character handling first (case, spaces, special chars).

TIME COMPLEXITY GUIDE

O(1) → Constant
O(log n) → Logarithmic
O(n) → Linear

1,966 822 125

Liked by _.lalit._ and others

abhi_techhub 🔥 Preparing DSA for 5–20 LPA interviews? Don't solve 500 random questions. Master the patterns that keep repeating.

3:25 ··· 90.7 KB/s 4G+ 46%

Instagram

abhi_techhub
KALEO · Way down We Go

insta @abhi_techhub.

MOST ASKED DSA INTERVIEW QUESTIONS

(0-3 YEARS) (5-20 LPA) PAGE 5

5/15

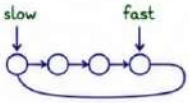
5. LINKED LIST - MOST ASKED QUESTIONS

★ Linked List questions test pointer manipulation and edge case handling.
Fast & Slow pointer technique is used in many problems.

#	QUESTION	PATTERN	EXPECTED COMPLEXITY	KEY IDEA / HINT
1	Reverse Linked List (LC 206)	Iterative (Three Pointers)	O(n) time O(1) space	• Keep prev, curr, next • Reverse the links one by one • Return prev at the end
2	Linked List Cycle (LC 141)	Fast & Slow Pointers (Floyd's Algo)	O(n) time O(1) space	• Move slow by 1, fast by 2 • If they meet, cycle exists • If fast reaches end, no cycle
3	Middle of Linked List (LC 876)	Fast & Slow Pointers	O(n) time O(1) space	• Move slow by 1, fast by 2 • When fast reaches end, slow will be at middle
4	Merge Two Sorted Lists (LC 21)	Two Pointers	O(n + m) time O(1) space	• Compare nodes of both lists • Attach smaller node to result • Move pointer of that list
5	Remove Nth Node From End (LC 19)	Two Pointers (Gap Technique)	O(n) time O(1) space	• Move fast n steps ahead • Move both until fast reaches end • Delete slow.next
6	Intersection of Two Linked Lists (LC 160)	Two Pointers (Switch Lists)	O(n + m) time O(1) space	• Traverse both lists • Switch head when you reach end • They will meet at intersection or become NULL

★ FAST & SLOW POINTER - WHEN TO USE?

Use Fast & Slow when:
- ✓ Detecting cycle in Linked List
- ✓ Finding middle element
- ✓ Finding start of cycle
- ✓ Finding k-th node from end

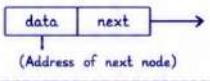


Why it works?
- ✓ Fast moves 2x, slow moves 1x
- ✓ If cycle exists, fast will lap slow
- ✓ If no cycle, fast reaches NULL

COMMON LINKED LIST OPERATIONS COMPLEXITY

- Traverse → O(n)
- Search → O(n)
- Insert at Head → O(1)
- Delete at Head → O(1)
- Insert at Tail → O(n)
- Delete at Tail → O(n)
- Insert at Position → O(n)
- Delete at Position → O(n)

NODE STRUCTURE



(Address of next node)

1,966 822 125

Liked by _.lalit._ and others

abhi_techhub 🔥 Preparing DSA for 5–20 LPA interviews? Don't solve 500 random questions. Master the patterns that keep repeating.

3:25 ··· 10.9 KB/s 4G+ 46%

Instagram

abhi_techhub
KALEO · Way down We Go

insta @abhi_techhub.

MOST ASKED DSA INTERVIEW QUESTIONS

(0-3 YEARS) (5-20 LPA) PAGE 3

3. TWO POINTERS + SLIDING WINDOW - MOST ASKED QUESTIONS

★ Two Pointers solve problems on sorted / unsorted arrays efficiently.
Sliding Window is used for subarray / substring problems.

#	QUESTION	PATTERN	EXPECTED COMPLEXITY	KEY IDEA / HINT
1	Container With Most Water (LC 11)	Two Pointers (Opposite Ends)	O(n) time O(1) space	• Move the pointer at smaller height • Area = $\min(h_1, h_2)$ x width
2	3Sum (LC 15)	Two Pointers + Sorting	$O(n^2)$ time O(1) space (extra for sort)	• Sort the array • Fix one element • Use two pointers for remaining
3	Remove Duplicates from Sorted Array (LC 26)	Two Pointers	O(n) time O(1) space	• Slow pointer for unique position • Fast pointer to scan • Copy when new unique found
4	Longest Substring Without Repeating Characters (LC 3)	Sliding Window + Hashing	O(n) time O(k) space (k = charset size)	• Expand window to include • Shrink window on duplicate • Keep max length
5	Maximum Sum Subarray of Size K (LC 643)	Sliding Window	O(n) time O(1) space	• Calculate sum of first window • Slide window, add new, remove old • Track max sum
6	Minimum Window Substring (LC 76)	Sliding Window + Hashing	O(n) time O(σ) space (σ = unique chars)	• Expand to include all chars • Shrink to minimum valid window • Track min length window

★ WHEN TO USE WHAT?

→ Use TWO POINTERS when:
- You need pairs/triplets
- Array is sorted / can be sorted
- Problem asks for min/max with two values

→ Use SLIDING WINDOW when:
- Subarray / substring is involved
- Need longest/shortest with some condition
- Elements are positive / count based

TWO POINTERS TEMPLATE

```
left = 0, right = n-1
while left < right:
    if condition:
        do something
```

Move pointers carefully and think

SLIDING WINDOW TEMPLATE

```
left = 0
for right in range(n):
    add nums[right] / expand
    while condition not valid:
```

1,966 822 125

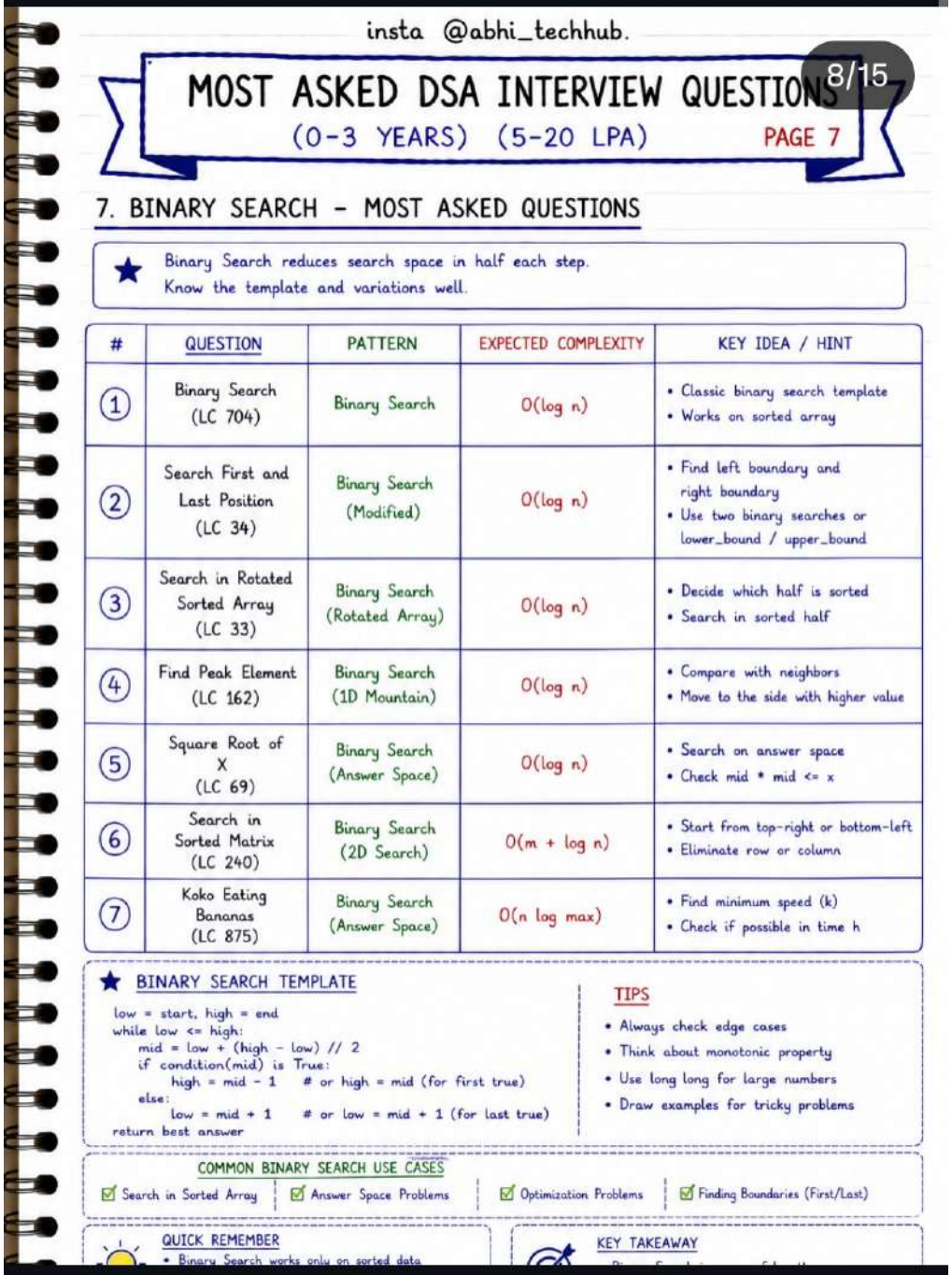
Liked by _.lalit._ and others

abhi_techhub 🔥 Preparing DSA for 5–20 LPA interviews? Don't solve 500 random questions. Master the patterns that keep repeating.

3:25 ··· 89.3 KB/s 4G+ 46%

Instagram

abhi_techhub
KALEO · Way down We Go



1,966 822 125

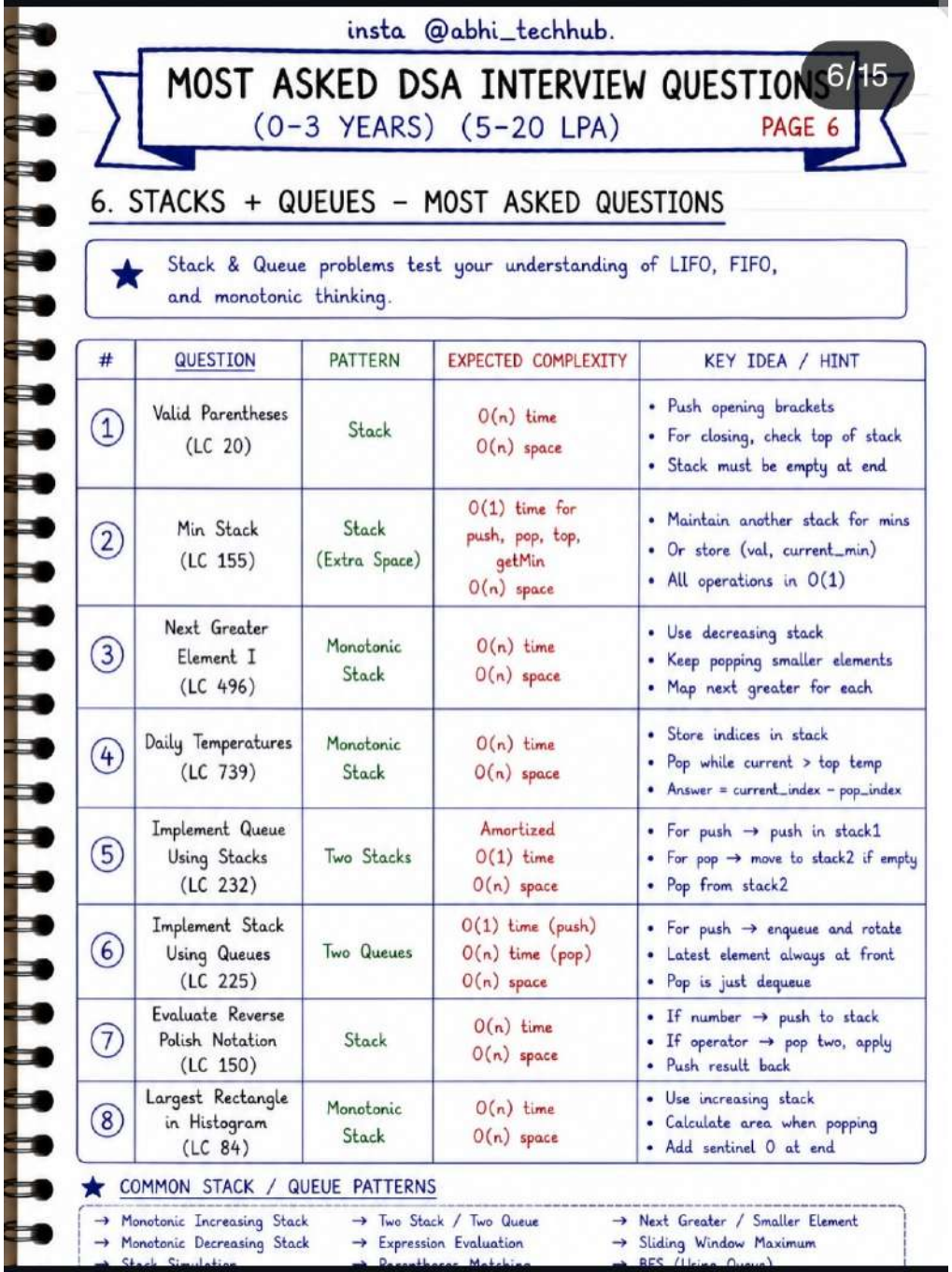
Liked by _.lalit._ and others

abhi_techhub 🔥 Preparing DSA for 5–20 LPA interviews? Don't solve 500 random questions. Master the patterns that keep repeating.

3:25 ··· 90.7 KB/s 4G+ 46%

Instagram

abhi_techhub
KALEO · Way down We Go



1,966 822 125

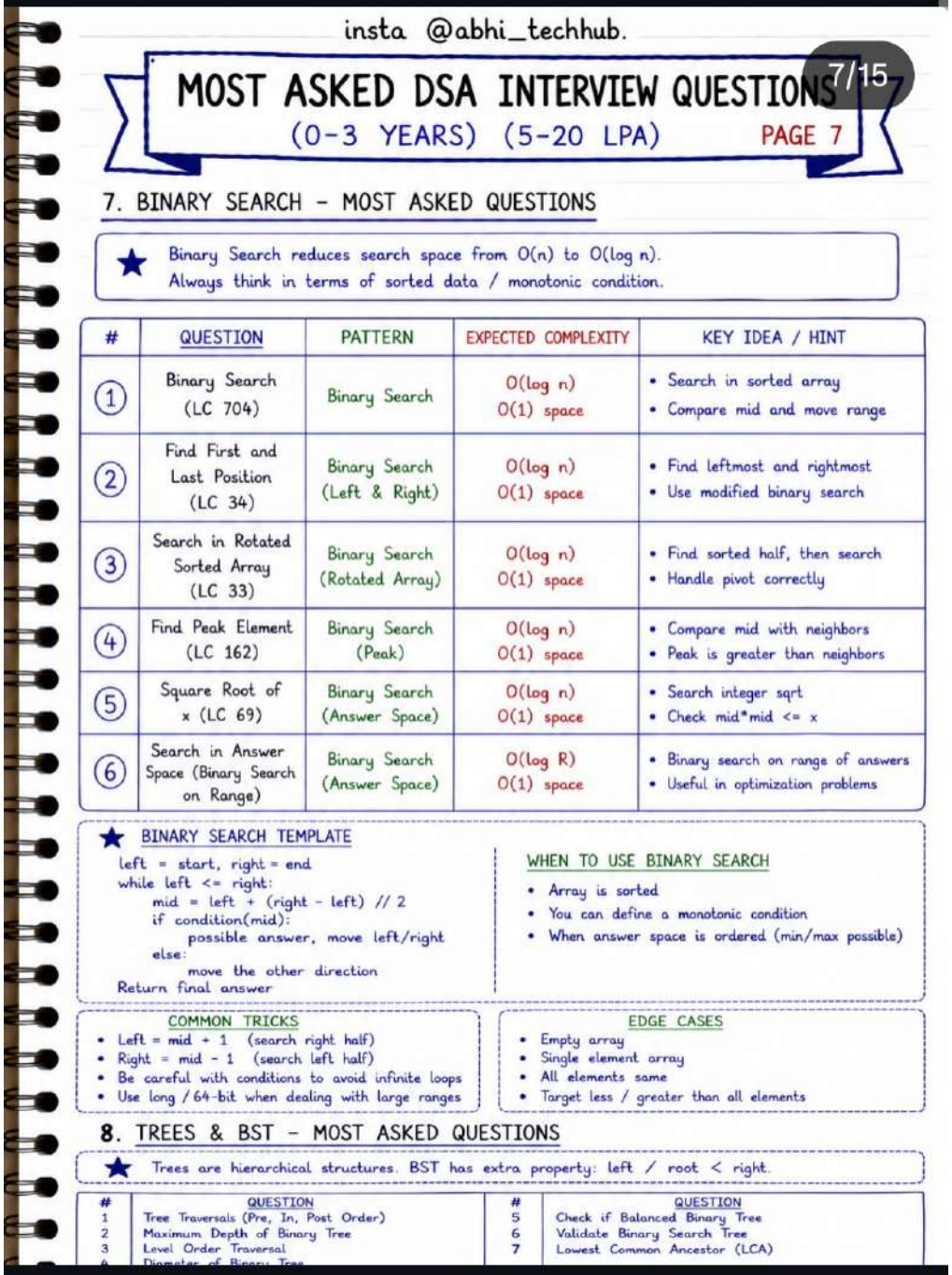
Liked by _.lalit._ and others

abhi_techhub 🔥 Preparing DSA for 5–20 LPA interviews? Don't solve 500 random questions. Master the patterns that keep repeating.

3:25 ··· 213 KB/s 4G+ 46%

Instagram

abhi_techhub
KALEO · Way down We Go



1,966 822 125

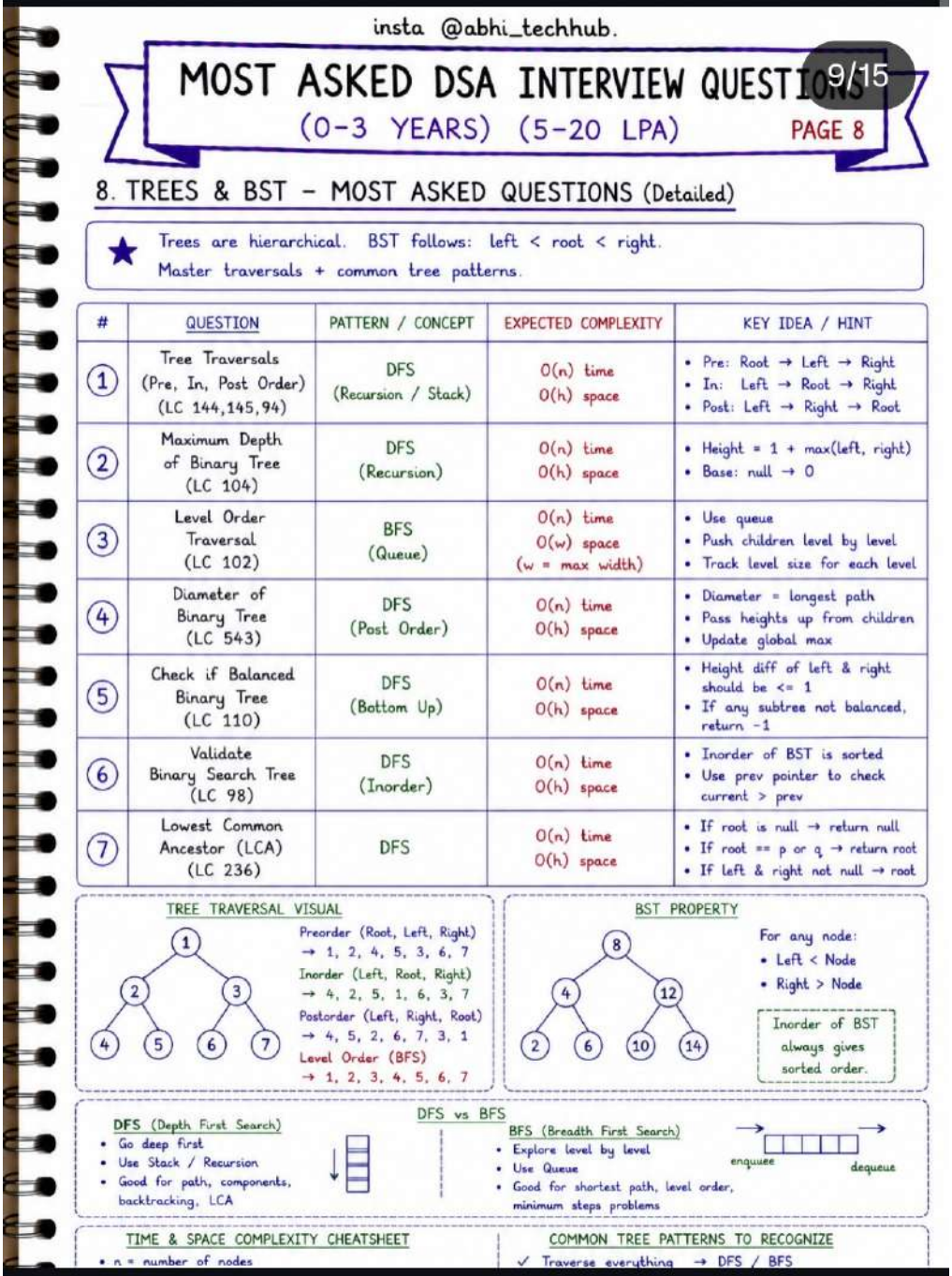
Liked by _.lalit._ and others

abhi_techhub 🔥 Preparing DSA for 5–20 LPA interviews? Don't solve 500 random questions. Master the patterns that keep repeating.

3:26 ··· 89.3 KB/s 4G+ 46%

Instagram

abhi_techhub
AI info



1,966 822 125

Liked by _.lalit._ and others

abhi_techhub 🔥 Preparing DSA for 5–20 LPA interviews? Don't solve 500 random questions. Master the patterns that keep repeating.

3:26 ··· 191 KB/s 4G+ 46%

Instagram

abhi_techhub
♫ KALEO · Way down We Go

insta @abhi_techhub.

MOST ASKED DSA INTERVIEW QUESTIONS

(0-3 YEARS) (5-20 LPA) PAGE 9

10/15

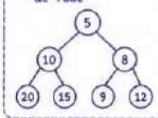
9. HEAPS (PRIORITY QUEUE) - MOST ASKED QUESTIONS

★ Heap is a complete binary tree that satisfies the heap property.
Used to implement Priority Queue efficiently.

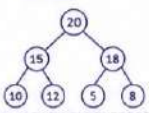
#	QUESTION	PATTERN / CONCEPT	EXPECTED COMPLEXITY	KEY IDEA / HINT
1	Kth Largest Element in Array (LC 215)	Heap (Min Heap)	O(n log k) time O(k) space	• Use Min Heap of size k • Push first k elements • For remaining, replace if greater • Top gives kth largest
2	K Closest Points to Origin (LC 973)	Heap (Max Heap)	O(n log k) time O(k) space	• Use Max Heap of size k • Push first k points by distance • Replace if new point is closer
3	Merge K Sorted Lists (LC 23)	Heap (Min Heap)	O(N log k) time O(k) space (N = total nodes)	• Push head of all lists in Min Heap • Pop min, push next of that list • Repeat until heap empty
4	Find Median from Data Stream (LC 295)	Two Heaps (Max + Min)	O(log n) time O(n) space	• Max Heap for smaller half • Min Heap for larger half • Balance sizes, get median
5	Top K Frequent Elements (LC 347)	Heap (Min Heap)	O(n log k) time O(k) space	• Count frequency using Map • Push (freq, element) in Min Heap • Keep size = k
6	Sort Nearly Sorted Array (LC 1046)	Heap (Min Heap)	O(n log k) time O(k) space	• Elements are at most k away • Use Min Heap of size k + 1 • Pop min and push next

HEAP TYPES

MIN HEAP
- Parent ≤ Child
- Smallest element at root



MAX HEAP
- Parent ≥ Child
- Largest element at root



HEAP OPERATIONS COMPLEXITY

Insert (push)	O(log n)
Delete Top (pop)	O(log n)
Peek / Top	O(1)
Build Heap	O(n)
Heapify	O(log n)

WHEN TO USE HEAP?

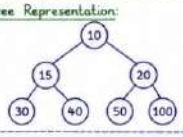
- K largest / smallest elements
- Merge K sorted lists
- Priority based processing
- Streaming median
- Scheduling / Simulation
- Dijkstra's Algorithm
- Huffman Coding

HEAP IMPLEMENTATION (ARRAY REPRESENTATION)

For node at index i (0-indexed):
Left Child = 2i + 1
Right Child = 2i + 2
Parent = (i - 1) / 2

Index:	0	1	2	3	4	5	6
Value:	10	15	20	30	40	50	100

Tree Representation:



HEAPIFY (MIN HEAP) - DOWN HEAP

QUICK TIPS

1,966 822 125

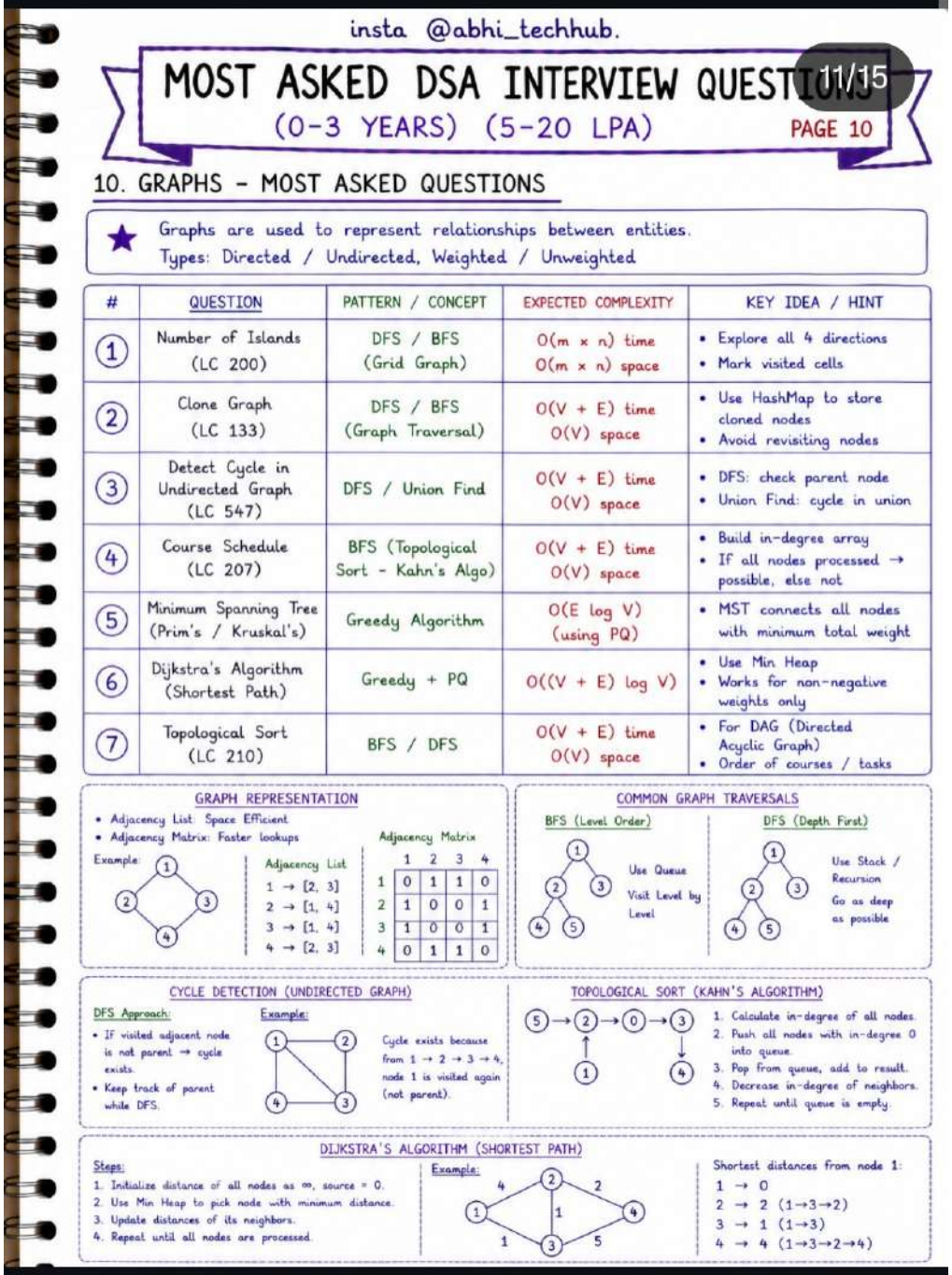
Liked by _.lalit._ and others

abhi_techhub 🔥 Preparing DSA for 5–20 LPA interviews? Don't solve 500 random questions. Master the patterns that keep repeating.

3:26 ··· 191 KB/s 4G+ 46%

Instagram

abhi_techhub
AI info



insta @abhi_techhub.

MOST ASKED DSA INTERVIEW QUESTIONS

(0-3 YEARS) (5-20 LPA) PAGE 10

11/15

10. GRAPHS - MOST ASKED QUESTIONS

★ Graphs are used to represent relationships between entities.
Types: Directed / Undirected, Weighted / Unweighted

#	QUESTION	PATTERN / CONCEPT	EXPECTED COMPLEXITY	KEY IDEA / HINT
1	Number of Islands (LC 200)	DFS / BFS (Grid Graph)	O(m x n) time O(m x n) space	• Explore all 4 directions • Mark visited cells
2	Clone Graph (LC 133)	DFS / BFS (Graph Traversal)	O(V + E) time O(V) space	• Use HashMap to store cloned nodes • Avoid revisiting nodes
3	Detect Cycle in Undirected Graph (LC 547)	DFS / Union Find	O(V + E) time O(V) space	• DFS: check parent node • Union Find: cycle in union
4	Course Schedule (LC 207)	BFS (Topological Sort - Kahn's Algo)	O(V + E) time O(V) space	• Build in-degree array • If all nodes processed → possible, else not
5	Minimum Spanning Tree (Prim's / Kruskal's)	Greedy Algorithm	O(E log V) (using PQ)	• MST connects all nodes with minimum total weight
6	Dijkstra's Algorithm (Shortest Path)	Greedy + PQ	O((V + E) log V)	• Use Min Heap • Works for non-negative weights only
7	Topological Sort (LC 210)	BFS / DFS	O(V + E) time O(V) space	• For DAG (Directed Acyclic Graph) • Order of courses / tasks

GRAPH REPRESENTATION

- Adjacency List: Space Efficient
- Adjacency Matrix: Faster lookups

Example:

Adjacency List
1 → [2, 3]
2 → [1, 4]
3 → [1, 4]
4 → [2, 3]

Adjacency Matrix

	1	2	3	4
1	0	1	1	0
2	1	0	0	1
3	1	0	0	1
4	0	1	1	0

COMMON GRAPH TRAVERSALS

BFS (Level Order)

Use Queue
Visit Level by Level

DFS (Depth First)

Use Stack / Recursion
Go as deep as possible

CYCLE DETECTION (UNDIRECTED GRAPH)

DFS Approach:

- If visited adjacent node is not parent → cycle exists.
- Keep track of parent while DFS.

Example:

Cycle exists because from 1 → 2 → 3 → 4, node 1 is visited again (not parent).

TOPOLOGICAL SORT (KAHN'S ALGORITHM)

1. Calculate in-degree of all nodes.
2. Push all nodes with in-degree 0 into queue.
3. Pop from queue, add to result.
4. Decrease in-degree of neighbors.
5. Repeat until queue is empty.

DIJKSTRA'S ALGORITHM (SHORTEST PATH)

Steps:

1. Initialize distance of all nodes as ∞, source = 0.
2. Use Min Heap to pick node with minimum distance.
3. Update distances of its neighbors.
4. Repeat until all nodes are processed.

Example:

Shortest distances from node 1:
1 → 0
2 → 2 (1→3→2)
3 → 1 (1→3)
4 → 4 (1→3→2→4)

1,966 822 125

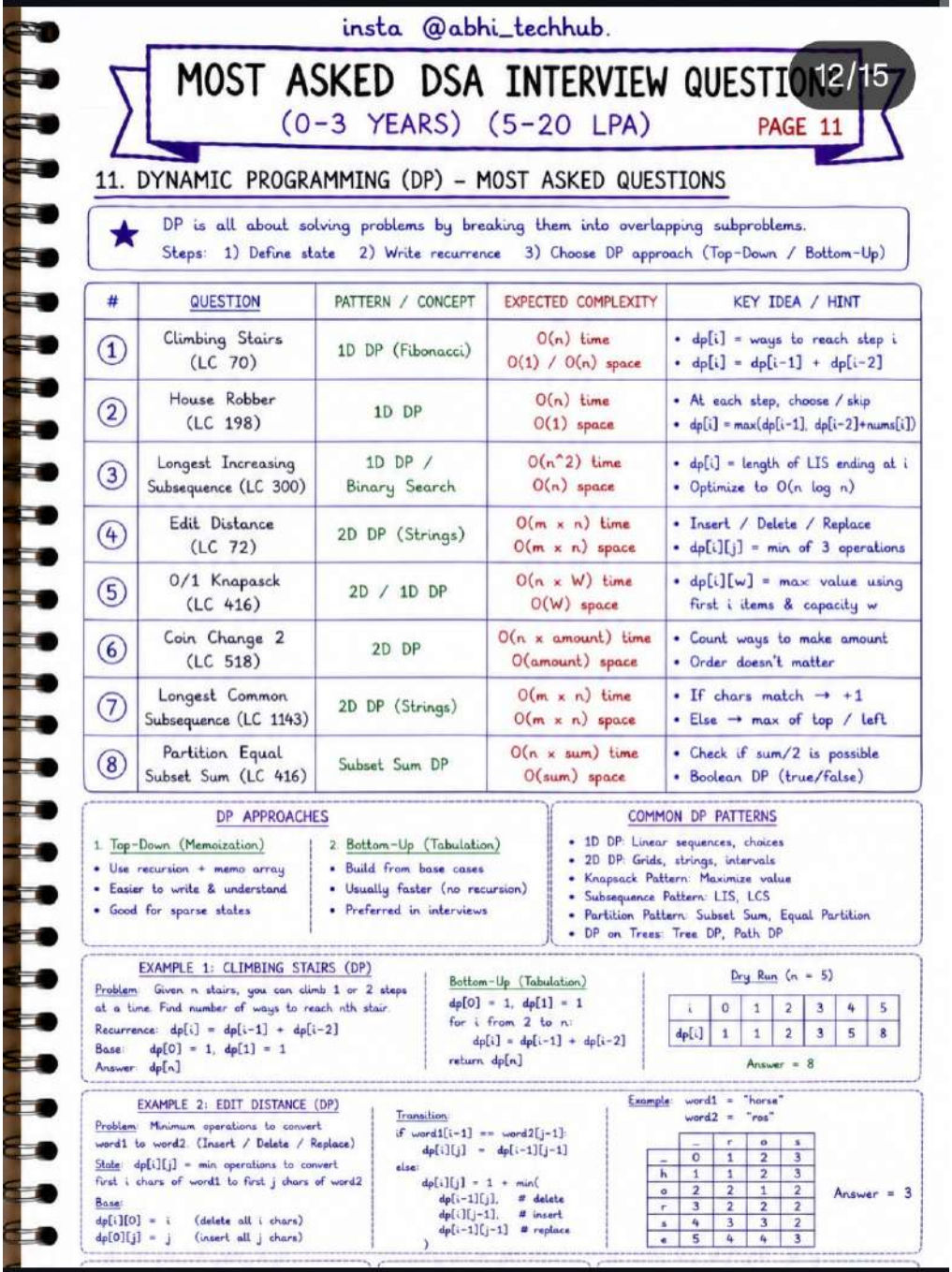
Liked by _.lalit._ and others

abhi_techhub 🔥 Preparing DSA for 5–20 LPA interviews? Don't solve 500 random questions. Master the patterns that keep repeating.

3:26 ··· 211 KB/s 4G+ 46%

Couldn't save screenshot
Try taking screenshot again

abhi_techhub
KALEO · Way down We Go



insta @abhi_techhub.

MOST ASKED DSA INTERVIEW QUESTIONS

(0-3 YEARS) (5-20 LPA) PAGE 11

12/15

11. DYNAMIC PROGRAMMING (DP) - MOST ASKED QUESTIONS

★ DP is all about solving problems by breaking them into overlapping subproblems.
Steps: 1) Define state 2) Write recurrence 3) Choose DP approach (Top-Down / Bottom-Up)

#	QUESTION	PATTERN / CONCEPT	EXPECTED COMPLEXITY	KEY IDEA / HINT
1	Climbing Stairs (LC 70)	1D DP (Fibonacci)	O(n) time O(1) / O(n) space	• dp[i] = ways to reach step i • dp[i] = dp[i-1] + dp[i-2]
2	House Robber (LC 198)	1D DP	O(n) time O(1) space	• At each step, choose / skip • dp[i] = max(dp[i-1], dp[i-2]+nums[i])
3	Longest Increasing Subsequence (LC 300)	1D DP / Binary Search	O(n^2) time O(n) space	• dp[i] = length of LIS ending at i • Optimize to O(n log n)
4	Edit Distance (LC 72)	2D DP (Strings)	O(m x n) time O(m x n) space	• Insert / Delete / Replace • dp[i][j] = min of 3 operations
5	0/1 Knapasck (LC 416)	2D / 1D DP	O(n x W) time O(W) space	• dp[i][w] = max value using first i items & capacity w
6	Coin Change 2 (LC 518)	2D DP	O(n x amount) time O(amount) space	• Count ways to make amount • Order doesn't matter
7	Longest Common Subsequence (LC 1143)	2D DP (Strings)	O(m x n) time O(m x n) space	• If chars match → +1 • Else → max of top / left
8	Partition Equal Subset Sum (LC 416)	Subset Sum DP	O(n x sum) time O(sum) space	• Check if sum/2 is possible • Boolean DP (true/false)

DP APPROACHES

1. Top-Down (Memoization)
- Use recursion + memo array
- Easier to write & understand
- Good for sparse states

2. Bottom-Up (Tabulation)
- Build from base cases
- Usually faster (no recursion)
- Preferred in interviews

COMMON DP PATTERNS

- 1D DP: Linear sequences, choices
- 2D DP: Grids, strings, intervals
- Knapsack Pattern: Maximize value
- Subsequence Pattern: LIS, LCS
- Partition Pattern: Subset Sum, Equal Partition
- DP on Trees: Tree DP, Path DP

EXAMPLE 1: CLIMBING STAIRS (DP)

Problem: Given n stairs, you can climb 1 or 2 steps at a time. Find number of ways to reach nth stair.
Recurrence: dp[i] = dp[i-1] + dp[i-2]
Base: dp[0] = 1, dp[1] = 1
Answer: dp[n]

Bottom-Up (Tabulation)

```
dp[0] = 1, dp[1] = 1
for i from 2 to n:
    dp[i] = dp[i-1] + dp[i-2]
return dp[n]
```

Dry Run (n = 5)

i	0	1	2	3	4	5
dp[i]	1	1	2	3	5	8

Answer = 8

EXAMPLE 2: EDIT DISTANCE (DP)

Problem: Minimum operations to convert word1 to word2. (Insert / Delete / Replace)
State: dp[i][j] = min operations to convert first i chars of word1 to first j chars of word2
Base:
dp[i][0] = i (delete all i chars)
dp[0][j] = j (insert all j chars)

Transition:

```
if word1[i-1] == word2[j-1]:
    dp[i][j] = dp[i-1][j-1]
else:
    dp[i][j] = 1 + min(
        dp[i-1][j],   # delete
        dp[i][j-1],   # insert
        dp[i-1][j-1]  # replace
    )
```

Example: word1 = "horse"
word2 = "ros"

	_	r	o	s
_	0	1	2	3
h	1	1	2	3
o	2	2	1	2
r	3	2	2	2
s	4	3	3	2
e	5	4	4	3

Answer = 3

1,966 822 125

Liked by _.lalit._ and others

abhi_techhub 🔥 Preparing DSA for 5–20 LPA interviews? Don't solve 500 random questions. Master the patterns that keep repeating.

3:26 ··· 4.34 MB/s 4G+ 46%

Instagram

abhi_techhub
AI info

insta @abhi_techhub.

MOST ASKED DSA INTERVIEW QUESTIONS

(0-3 YEARS) (5-20 LPA) PAGE 12

12. GRAPH ALGORITHMS - MOST ASKED QUESTIONS

★ Graph algorithms help us find shortest path, minimum cost, connectivity, components etc. Know when to use which!

#	QUESTION	ALGORITHM	EXPECTED COMPLEXITY	KEY IDEA / HINT
1	Shortest Path in Unweighted Graph (LC 1091)	BFS	O(V + E) time O(V) space	• BFS from source • First time we visit = shortest
2	Shortest Path in Weighted Graph (LC 743)	Dijkstra's Algorithm (Min Heap)	O((V + E) log V) time O(V) space	• Non-negative weights • Greedy: always pick minimum distance node
3	Detect Negative Cycle (LC 209)	Bellman-Ford Algorithm	O(V × E) time O(V) space	• Relax all edges V-1 times • One more relaxation that improves → cycle
4	Find All Bridges in Graph (LC 1192)	Tarjan's Algorithm (DFS + Low Time)	O(V + E) time O(V) space	• An edge (u,v) is bridge if low[v] > disc[u]
5	Find Articulation Points (LC 1125)	Tarjan's Algorithm (DFS + Low Time)	O(V + E) time O(V) space	• Vertex whose removal increases no. of connected components
6	Minimum Spanning Tree (MST) (LC 1584)	Kruskal's / Prim's Algorithm	O(E log E) or O((V + E) log V)	• MST connects all nodes with minimum total weight
7	Check if Graph is Bipartite (LC 785)	BFS / DFS (Coloring)	O(V + E) time O(V) space	• Try to color using 2 colors • If conflict → not bipartite

SHORTEST PATH - QUICK GUIDE

- Unweighted Graph → BFS
- Weighted (non-negative) → Dijkstra
- Negative weights → Bellman-Ford
- Need only yes/no reachability → DFS/BFS

DIJKSTRA (MIN HEAP) - IDEA

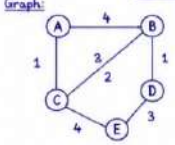
- Keep distance[] array
- Min Heap stores (dist, node)
- Pop node with smallest dist
- Relax all its neighbours

BELLMAN-FORD - IDEA

- Relax all edges V-1 times
- If any edge can still be relaxed on Vth time → Negative Cycle
- Works with negative weights

DIJKSTRA EXAMPLE

Graph:

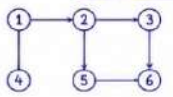


Source = A
Steps (dist[]):
Init: A=0, B=∞, C=∞, D=∞, E=∞
Pick A(0) → relax: B=4, C=1
Pick C(1) → relax: B=3, E=5
Pick B(3) → relax: D=4
Pick D(4) → relax: E=5 (unchanged)
Pick E(5) → done
Final dist: A=0, B=3, C=1, D=4, E=5

KRUSKAL vs PRIM

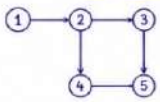
Kruskal (Edge based)	Prim (Node based)
• Sort all edges	• Start from any node
• Add smallest edge	• Grow MST by adding min edge to new node
• If it doesn't form cycle	• Use Min Heap
• Use Disjoint Set (DSU)	

TARJAN - BRIDGE FINDING



Edge (u,v) is a bridge if:
low[v] > disc[u]
Here, edges (1,4) and (3,6) are bridges.

TARJAN - ARTICULATION POINTS



Node u is articulation point if:
low[v] ≥ disc[u]
(v is child of u in DFS tree)
Here, nodes 2 and 3 are articulation points.

GRAPH ALGORITHMS CHEATSHEET

1,966 822 125

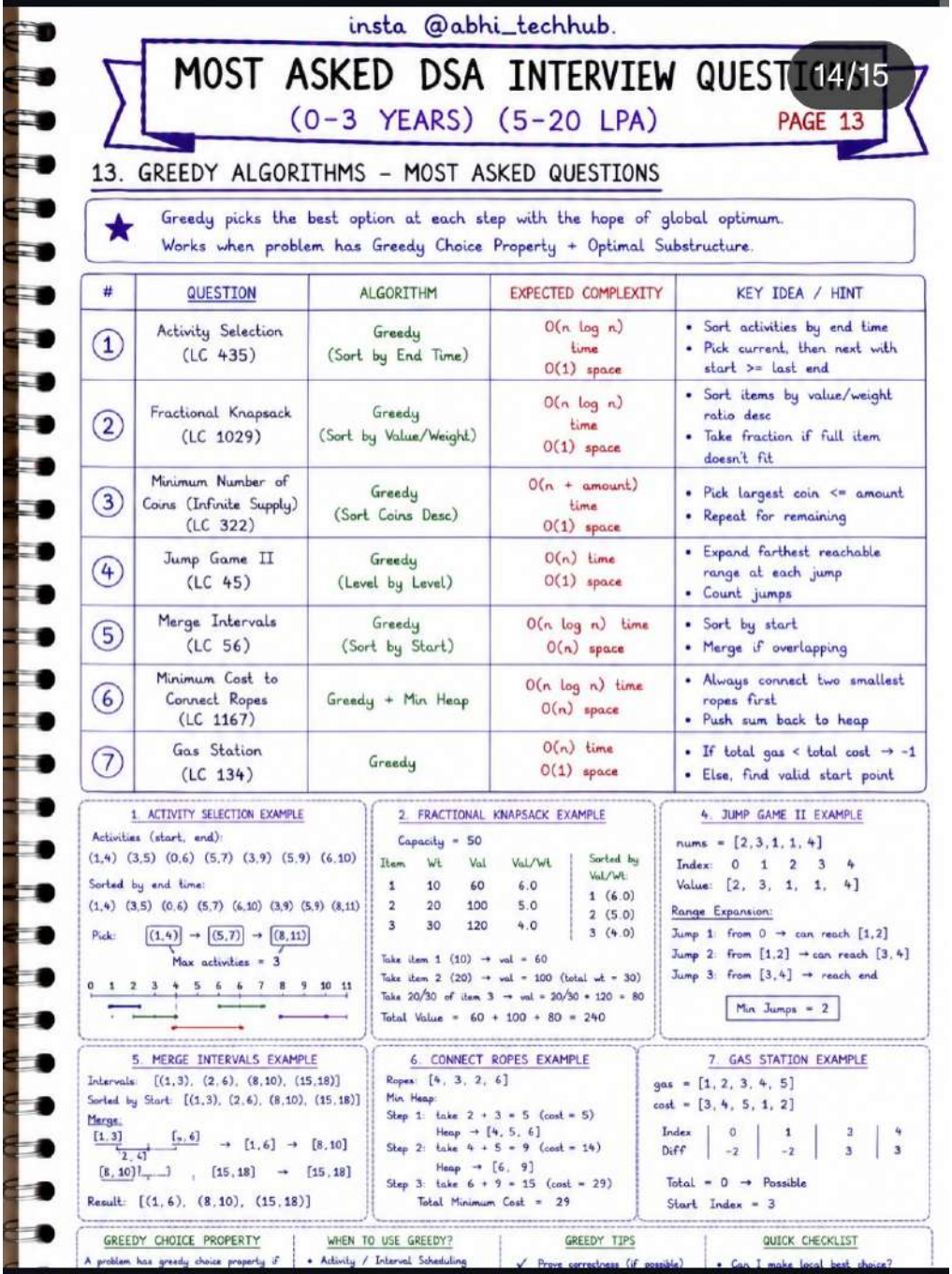
Liked by _.lalit._ and others

abhi_techhub 🔥 Preparing DSA for 5–20 LPA interviews? Don't solve 500 random questions. Master the patterns that keep repeating.

3:26 ··· 3.78 MB/s 4G+ 46%

Instagram

abhi_techhub
KALEO · Way down We Go



1,966 822 125

Liked by _.lalit._ and others

abhi_techhub 🔥 Preparing DSA for 5–20 LPA interviews? Don't solve 500 random questions. Master the patterns that keep repeating.

3:27 ··· 5.48 MB/s 4G+ 46%

Instagram

akanksha_buchke
KREZUS, Surreal_dvd · Skins

1/4

40 hand-picked interview questions from Arrays, Strings & HashMap for service-based companies (TCS, Infosys, Accenture, Wipro, Cognizant, Capgemini, etc.)

⭐Arrays (15 Questions)

1. Find largest & smallest element in array
2. Second largest element
3. Check if array is sorted
4. Left rotate array by K
5. Move all zeros to end
6. Remove duplicates from sorted array
7. Find missing number (1 to n)
8. Two Sum
9. Three Sum (basic version)
10. Maximum subarray sum - Kadane
11. Product of array except self
12. Find duplicate element
13. Merge two sorted arrays
14. Container with most water
15. Trapping rain water

1,866 1,920 125

akanksha_buchke Comment “List” for PDF 📝

Preparing for service-based company interviews?

These 40 hand-picked Array, String, and HashMap questions are enough to boost your coding

3:27 ··· 5.48 MB/s 4G+ 46%

Instagram

akanksha_buchke
♫ KREZUS, Surreal_dvd · Skins

2/4

⭐ STRINGS (15 Questions)

1. Reverse a string
2. Check palindrome
3. Count vowels & consonants
4. Remove spaces from string
5. Convert uppercase lowercase
6. Check anagram
7. Remove duplicates from string
8. First non-repeating character
9. Frequency of characters using map
10. Longest substring without repeating chars
11. Check rotation of string
12. String compression
13. Valid parentheses
14. Longest common prefix
15. Check if two strings are isomorphic

1,866 1,920 125

akanksha_buchke Comment “List” for PDF 📝

Preparing for service-based company interviews?

These 40 hand-picked Array, String, and HashMap questions are enough to boost your coding

3:27 ··· 4.54 MB/s 4G+ 46%

Instagram

akanksha_buchke
♫ KREZUS, Surreal_dvd · Skins

⭐HASHMAP (10 Questions)

1. Count frequency of elements
2. Find duplicates using hashmap
3. Two Sum using map
4. First repeating element
5. Subarray with sum K
6. Longest subarray with sum 0
7. Count distinct elements in window
8. Group anagrams
9. Intersection of two arrays using map
10. Majority element using hashmap

Follow and comment "List" for PDF🙌🏻

1,866 1,920 125

akanksha_buchke Comment "List" for PDF 📝

Preparing for service-based company interviews?

These 40 hand-picked Array, String, and HashMap questions are enough to boost your coding